

StudentPosts

Javadoc-style package documentation for the TP2 discussion-board MVC implementation

1. Overview

Task 4 asks for a set of appropriately named MVC GUI packages that implement the complete students' discussion-post user stories by employing the Post class from Task 3. The package set is centered on `guiDiscussionBoard`, which introduces a feature-specific JavaFX discussion board and separates view construction from controller logic while still fitting the existing Foundations project structure.

The package uses `Post` as the primary entity for discussion threads, `Reply` for child responses, `CreateInfo` for shared metadata, and the existing `Database / FoundationsMain` infrastructure for persistence and session context. `ModelDiscussionBoard` is currently a lightweight placeholder so the feature still presents a recognizable MVC trio and can absorb future view-state responsibilities without renaming the package structure.

2. Package summary

Package / Area	Primary contents	Why it exists
guiDiscussionBoard	<code>ViewDiscussionBoard</code> , <code>ControllerDiscussionBoard</code> , <code>ModelDiscussionBoard</code>	Holds all discussion-board GUI logic in one feature-scoped package named consistently with the rest of the Foundations GUI code.
entityClasses (dependency)	<code>Post</code> , <code>Reply</code> , <code>CreateInfo</code> , <code>User</code>	Provides the entities and metadata consumed by the discussion-board package. <code>Post</code> is the anchor class required by Task 3.
database / applicationMain (dependency)	<code>Database</code> , <code>FoundationsMain.database</code>	Supplies persistence, current-user identity, and the existing application-level entry points already present in the project.

3. MVC implementation at a glance

The uploaded code follows a practical MVC split for this feature:

- `ViewDiscussionBoard` constructs windows, tables, buttons, search/filter controls, and the single-post detail view.
- `ControllerDiscussionBoard` receives UI-triggered actions and performs CRUD orchestration, validation, permission checks, and database calls.
- `ModelDiscussionBoard` preserves the MVC structural slot for the feature even though durable state is currently represented by `Post`, `Reply`, and `Database` rather than a separate in-package model object.

4. Package detail: guiDiscussionBoard

```
package guiDiscussionBoard;
```

Purpose: to provide a complete GUI feature for student discussion posts, including the post-list screen, a full single-post view, reply handling, permission-aware actions, and lightweight search/status filtering.

Naming rationale: the package follows the same feature-scoped gui... naming pattern already used in the Foundations codebase, which keeps discovery and maintenance straightforward when additional GUI features are added.

Class summary

Class	Role in MVC	Summary of contribution to the student stories
ViewDiscussionBoard	View	Builds the JavaFX screens that let students list, view, filter, create, edit, delete, and resolve posts or replies.
ControllerDiscussionBoard	Controller	Bridges the view to Post/Reply/Database by enforcing authorization checks, dialogs, confirmation prompts, validation, and persistence updates.
ModelDiscussionBoard	Model placeholder	Maintains architectural alignment with MVC and leaves a clear expansion point for future derived state, caching, or presentation-state helpers.

5. Class detail: ControllerDiscussionBoard

```
public class ControllerDiscussionBoard
```

Purpose: central action coordinator for the discussion-board feature. The view instantiates this controller and delegates all CRUD-oriented operations to it so that validation, permission checks, and persistence stay out of the JavaFX layout code.

Field summary

Field	Type	Source / rationale
db	static Database	Bound to FoundationsMain.database so the controller can read persisted posts/replies and update the same application-wide database/session object used elsewhere in the project.
posts	ArrayList<Post>	In-memory working list for all discussion posts shown in the list view. This supports fast refreshes without forcing the view to understand database details.
replies	HashMap<String, ArrayList<Reply>>	Maps parent post IDs to reply lists. This is a natural structure for the single-post screen because replies are always accessed through a selected post.

Constructor and method summary

Member	Type	Behavior / why it matters
ControllerDiscussionBoard()	constructor	Loads posts and replies from the database when the feature starts so the UI opens with persisted data rather than empty placeholder state.
listPosts()	read	Supplies the current post collection to the view for initial display and later refreshes.
getPostReplies(String)	read	Returns the replies that belong to a selected post ID so the detail window can populate its replies table.
canEdit(Post,...), canEdit(Reply,...)	authorization	Implements the 'admin or original poster' rule and optionally raises an error dialog when a user tries to edit or delete something they do not own.
createPost()	create	Collects header/body input, enforces non-empty fields, caps lengths through the dialog helper, creates a Post, stores it in the database, and inserts it into the in-memory list.
editPost(Post)	update	Prefills existing values, rejects empty edits or unchanged submissions, updates the Post object, and pushes the update to the database.
deletePost(Post)	delete	Uses a confirmation dialog before removing the post from persistence, the local list, and the reply map.
toggleResolved(Post)	state change	Marks a post solved or re-opened and persists that status so the view can treat solved posts differently.
createReply(String)	create	Collects reply text, validates non-empty input, creates a Reply, persists it, and attaches it under the correct parent post.
editReply(Reply)	update	Rejects unchanged/empty edits and persists reply text changes.
deleteReply(Reply)	delete	Requires confirmation before removing the reply from the database and the in-memory reply list.

Requirement coverage

- Post CRUD is centered here: createPost, editPost, deletePost, plus listPosts for read/list behavior.
- Reply CRUD is also centered here: createReply, editReply, deleteReply, plus getPostReplies for read/list behavior.
- Input validation lives here at the user-facing layer through non-empty checks, unchanged-edit checks, max-length enforcement via custom dialogs, and informative Alert dialogs.
- Permission-sensitive behavior also lives here so the view is never trusted as the final gate for edit/delete operations.
- Solved / resolved thread behavior is coordinated here through toggleResolved(Post), which persists state changes back to the database.

6. Class detail: ViewDiscussionBoard

```
public class ViewDiscussionBoard
```

Purpose: construct and refresh the discussion-board interface. This class owns the JavaFX table definitions, layout containers, window sequencing, filter widgets, and event wiring. It delegates business logic to ControllerDiscussionBoard rather than embedding persistence logic directly in button handlers.

Field summary

Field	Type	Source / rationale
postsTable	TableView<Post>	Main list of discussion posts on the feature's landing window. This is the student's starting point for browse/read behavior.
repliesTable	TableView<Reply>	Replies list shown in the single-post detail window. Keeps thread reading and reply management attached to one selected post.
controller	ControllerDiscussionBoard	Dedicated controller instance used by every button handler so UI code stays thin.
activePost	Post	Tracks the post currently being displayed in the detail window.
label_Header	Label	Prominent title area for the selected post header and solved marker.
post_Body	TextArea	Read-only body display for the selected post; styled to remove the normal editable control chrome.
parentStage / viewStage	Stage	Separates the list window from the post-detail window while preserving modal ownership.
filters	HBox	Holds the live search field and solved-status combo box used to form subset views of the post collection.

Method summary

Member	Type	Behavior / why it matters
display(Stage)	entry point	Creates the feature and opens the top-level discussion window.
show(Stage)	view setup	Builds the main posts table, search/status controls, and create/view/edit/delete controls.
showPost(Post)	detail view	Creates the single-post window with header, body, replies table, reply controls, and resolved-state actions.
refreshReplies()	view refresh	Rebuilds the replies table contents for the active post.
viewPost(), selectedOrWarnPost(), selectedReply()	selection helpers	Support safe transitions from the main list into the detailed view and provide warning dialogs when the user has not made a required selection.
refresh()	subset/list logic	Builds or updates a FilteredList<Post> so students can see all posts or a filtered subset by solved status and body-text search.
onCreate, onEdit1, onDelete1, onSolve, onEdit2, onDelete2, onCreateReply	event handlers	Translate button clicks into controller calls and then refresh the visible tables or windows so the UI stays synchronized with the underlying objects.
setupLabelUI(...)	view helper	Keeps repeated label formatting code out of the higher-level view methods.

Why these design choices matter

- The main post table shows creation/edit metadata, poster name, header, solved marker, and summary/body text, which supports quick scanning before the user opens a thread.
- The detail view keeps the original post at the top and nests replies below it, which matches the mental model expected from Ed-style discussion systems.
- `refresh()` uses a `FilteredList` so the feature can produce subset views without mutating or losing the full list of posts.
- The reply table uses a custom `TableCell` with wrapped text so long replies remain readable inside the table instead of forcing horizontal scrolling.
- The view still calls `controller.canEdit(...)` before edit/delete actions, but the controller remains the final authorization gate.

7. Class detail: `ModelDiscussionBoard`

```
public class ModelDiscussionBoard
```

Purpose: architectural placeholder. The class is intentionally lightweight now, but it preserves the same MVC class trio used elsewhere in the application and provides an obvious home for future derived state such as cached filter settings, pagination, statistics, or view-model adapters.

8. Collaboration with Post and supporting entities

The GUI package does not duplicate the Post entity. Instead, it deliberately employs Post (from Task 3) as the primary thread object and adds only the GUI-specific behavior required to create, display, edit, delete, and solve those posts.

Dependency	How StudentPosts uses it	Why it matters
Post	Displayed in both the main table and detail window; created/edited/resolved/deleted by the controller.	Task 4 explicitly requires the GUI packages to employ the Post class from Task 3.
Reply	Used in the replies table and reply CRUD flows.	Student discussion boards need a separate child entity for thread responses.
CreateInfo	Indirectly surfaced through <code>getCreateInfoFormatted()</code> and related metadata display.	Gives the GUI accurate timestamps, creator identity, and edit markers without duplicating metadata fields in view code.
User / session context	Current identity is pulled through the database/session layer rather than passed manually through every method.	Needed for ownership checks and author tagging.
Database + FoundationsMain.database	Supplies persistence and current-user information.	Lets the feature plug into the existing Foundations application rather than becoming a disconnected prototype.
TwoInputDialog / TextAreaDialog	Encapsulate structured user input with size and max-length controls.	Keeps validation and prompt layout consistent with the rest of the GUI feature.

9. Student-story coverage

Requirement area	Where it appears in the GUI package	Notes
Create post	View: onCreate -> Controller: createPost()	Header/body prompts plus validation; successful creates refresh the post list.
Read/list posts	View: show(), refresh(), postsTable	Initial table view plus filtered subsets.
View one post	View: viewPost(), showPost(Post)	Opens a dedicated thread window with header, body, and replies.
Update post	View: onEdit1 / onEdit2 -> Controller: editPost(Post)	Supports editing from both the list screen and the detail screen.
Delete post	View: onDelete1 / onDelete2 -> Controller: deletePost(Post)	Confirmation dialog prevents accidental destructive actions.
Create reply	View: onCreateReply -> Controller: createReply(String)	Reply dialog plus validation; stored under the active post ID.
Read/list replies	View: showPost(), refreshReplies(), repliesTable	Replies display only in the context of their parent post.
Update reply	View: onEdit2 -> Controller: editReply(Reply)	Uses selected reply from the replies table.
Delete reply	View: onDelete2 -> Controller: deleteReply(Reply)	Confirmation-driven destructive action.
Search / subset behavior	View: filters + refresh()	Subset generation uses solved status plus body-text matching.
Solved / closed threads	View: showPost(), onSolve() and Controller: toggleResolved(Post)	Solved state changes available actions and updates the Post's solved flag.
Input validation + warnings	Controller validation branches + View selection warnings	Blank inputs, unchanged edits, missing selections, and unauthorized edits/deletes all surface informative dialogs.

10. Screenshot appendix

```
133
134●  /**
135   * Prompts user to create a new post then saves the new post to the database
136   * Input validation is handled internally, void if cancelled or invalid.
137   */
138●  public void createPost()
139  {
140      //Create a new Two Input Dialog
141      TwoInputDialog newPostDialog = new TwoInputDialog("New Discussion Post", "Create a new discussion post.", "Header", "Body");
142
143      //Setup some visual elements of the input
144      newPostDialog.changeTextfieldSizes(600, -1, 600, 300);
145      newPostDialog.setMaxCharsForFields(100, 1000); //No Input Validation needed as a max char count is forced here
146      newPostDialog.getTextfield2().setWrapText(true);
147
148
149      Optional<TwoStringResults> postCreateResults; //Optional to contain the results
150
151      postCreateResults = newPostDialog.showAndWait(); //Show the Two input dialog
152
153      //If the user input is not empty, or they did not hit cancel create new post
154●  if (postCreateResults.isPresent() && !postCreateResults.get().getText1().equals("") && !postCreateResults.get().getText2().equ
155  {
156      //Create new Post object
157      Post newPost = new Post(db.getCurrentUsername(), postCreateResults.get().getText1(), postCreateResults.get().getText2());
158
159      db.createPost(newPost); //Save to database
160      posts.addFirst(newPost); //Add to top of the list
161●  } else if (postCreateResults.isPresent())
162  {
163      Alert invalidInput = new Alert(AlertType.ERROR);
164      boolean header = postCreateResults.get().getText1().equals("");
165      boolean body = postCreateResults.get().getText2().equals("");
166      invalidInput.setHeaderText("Post Cannot Be Created");
167      invalidInput.setTitle("Invalid Operation");
168
169●  if (header & body)
170  {
171      invalidInput.setContentText("Your post must contain a header and a body");
172●  } else if (header)
173  {
174      invalidInput.setContentText("Your post must contain a header");
175●  } else if (body)
176  {
177      invalidInput.setContentText("Your post must contain a body");
178  }
179
180      invalidInput.showAndWait();
181  }
182  }
183
```

Post creation flow in ControllerDiscussionBoard: shows dialog setup, max-length enforcement, persistence, and the non-empty validation branch.


```

277● /**
278 * Warns the user of resolved status consequences then toggles the resolved status of a post when confirmed
279 * Updates both the object and the database
280 *
281 * @param post post object to be to change resolved status
282 */
283● public void toggleResolved(Post post)
284 {
285     //Alert for confirmation
286     Alert confirm = new Alert(AlertType.CONFIRMATION, (post.getSolved()) ? "This action will re-open the discussion for replies."
287
288     //Set alert content
289     confirm.setTitle((post.getSolved()) ? "Re-Open Discussion" : "Mark Resolved");
290     confirm.setHeaderText((post.getSolved()) ? "Re-Open Discussion?" : "Mark as Resolved?");
291
292     Optional<ButtonType> selection = confirm.showAndWait();
293
294●     if (selection.isPresent())
295     {
296         //If user presses yes change the status and update the post
297●         if (selection.get() == ButtonType.YES)
298         {
299             post.markAsSolved(!post.getSolved());
300             db.updatePostSolvedStatus(post);
301         }
302     }
303 }
304
305● /**
306 * Prompts user to create a new reply then saves the new post to the database
307 * Input validation is handled internally, void if cancelled or invalid.
308 *
309 * @param postId the unique identifier of the post which this reply is associated with
310 */
311● public void createReply(String postId)
312 {
313
314     //Create a text area dialog
315     TextAreaDialog replyDialog = new TextAreaDialog("Create New Reply", "Please Enter Your Reply", "Body");
316
317     //Set up dialog
318     replyDialog.changeTextAreaSize(500, 200);
319     replyDialog.getTextArea().setWrapText(true);
320     replyDialog.setMaxChars(300); //Force max chars
321
322     Optional<String> replyCreateResults = replyDialog.showAndWait(); //Results
323
324     //If they input a new reply that is not empty
325●     if(replyCreateResults.isPresent() && !replyCreateResults.get().equals(""))
326     {
327         Reply newReply = new Reply(db.getCurrentUser(), replyCreateResults.get(), postId);
328
329         //Save it to the database and add to relevant lists
330         db.createReply(newReply);
331         replies.putIfAbsent(postId, new ArrayList<Reply>());
332         replies.get(postId).add(newReply);
333●     }else if (replyCreateResults.isPresent())
334     {
335         Alert invalidInput = new Alert(AlertType.ERROR, "Your reply must contain text.");
336         invalidInput.setHeaderText("Reply Could Not Be Created");
337         invalidInput.setTitle("Invalid Operation");
338         invalidInput.showAndWait();
339     }
340 }
341

```

Resolved-thread state change and reply creation logic: documents how solved status is persisted and how reply creation is validated before being attached to a parent post.

```

354
355 /**
356  * Refreshes the list of posts based on the filter values.
357  * Creates and displays a new filtered list of values based on the selected filters
358  */
359 private void refresh() {
360
361     //Load Latest summaries list
362     List<Post> summaries = controller.listPosts();
363
364     TextField search = (TextField) filters.getChildren().get(1); //Get the search field
365     @SuppressWarnings("rawtypes")
366     ComboBox solvedFilter = (ComboBox) filters.getChildren().get(3); //Get the combo box
367
368     String q = (search.getText() == null ? "" : search.getText().trim().toLowerCase()); //If there is nothing then the filter is ""
369     String status = (String) solvedFilter.getValue(); // "All", "Solved", "Unsolved"
370
371     //If the table is set up already
372     if (postsTable.getItems() instanceof FilteredList)
373     {
374         FilteredList<Post> filtered = (FilteredList<Post>) postsTable.getItems();
375         @SuppressWarnings("unchecked")
376         ObservableList<Post> allPosts = (ObservableList<Post>) filtered.getSource();
377         allPosts.setAll(summaries); //Set the new list of items
378         filtered.setPredicate(p -> {
379
380             //Return false if p is null
381             if (p == null) return false;
382
383             //If the status matches
384             boolean statusMatch = "All".equals(status) || ("Solved".equals(status) && p.getSolved()) || ("Unsolved".equals(status)
385
386             //If the body of the post matches
387             boolean bodyMatch = q.isEmpty() || (p.getBody() != null && p.getBody().toLowerCase().contains(q));
388
389             //Return the result of the matches
390             return statusMatch && bodyMatch;
391         });
392

```

```

392
393     //If this is the first call of refresh
394     } else {
395
396         ObservableList<Post> allPosts = FXCollections.observableArrayList(summaries); //Create new list using list of posts
397         FilteredList<Post> filtered = new FilteredList<>(allPosts, p -> true); //Create a new filtered list using the
398         postsTable.setItems(filtered); //Set the table to the filtered list
399
400
401         //Set the predicate for the filtered list
402         filtered.setPredicate(p -> {
403
404             //Return false if p is null
405             if (p == null) return false;
406
407             //If the status matches
408             boolean statusMatch = "All".equals(status) || ("Solved".equals(status) && p.getSolved()) || ("Unsolved".equals(status)
409
410             //If the body of the post matches
411             boolean bodyMatch = q.isEmpty() || (p.getBody() != null && p.getBody().toLowerCase().contains(q));
412
413             //Return the result of the matches
414             return statusMatch && bodyMatch;
415         });
416     }
417 }
418

```

refresh() in ViewDiscussionBoard: demonstrates the subset/list requirement by building a FilteredList based on solved status and body-text search.

```

204 //Content
205 TableColumn<Reply, String> colBody = new TableColumn<>("Reply");
206 colBody.setCellValueFactory(c -> new ReadOnlyStringWrapper(c.getValue().getBody()));
207 colBody.setPrefWidth(400);
208
209 //This sets it so that the reply content will wrap so that it can be read better
210 //It creates a new type of table cell to be used for the content
211 colBody.setCellFactory(tc -> new TableCell<Reply, String>() {
212
213     //New Text
214     private final javafx.scene.text.Text text = new javafx.scene.text.Text();
215     {
216         text.wrappingWidthProperty().bind(colBody.widthProperty().subtract(16)); //Turn on Text Wrapping
217         setGraphic(text); //Set the cell graphic to text
218         setContentDisplay(ContentDisplay.GRAPHIC_ONLY);
219         setPrefHeight(Region.USE_COMPUTED_SIZE); //Compute the size required for the reply
220     }
221
222     //When needed cell is updated
223     @Override
224     protected void updateItem(String item, boolean empty) {
225         super.updateItem(item, empty); //Normal Cell Function
226
227         //If the item is empty or the string is empty set text to null
228         if (empty || item == null) {
229             text.setText(null);
230             setGraphic(null);
231
232             //Else update the graphic normally
233         } else {
234             text.setText(item);
235             setGraphic(text);
236         }
237     }
238 });
239
240
241
242 //Add all the replies to the table
243 repliesTable.getColumns().addAll(colDate, colName, colBody);
244 repliesTable.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY_FLEX_LAST_COLUMN); //This just makes it look better, it
245 repliesTable.setPrefHeight(150);
246 repliesTable.setPlaceholder(new Label("Be the first to reply?"));
247
248
249 // Bottom: actions
250 Button btnReply = new Button("Reply");
251 Button btnEdit = new Button("Edit");
252 Button btnDelete = new Button("Delete");
253 Button btnClose = new Button("Close");
254 Button btnMarkResolved = (activePost.getSolved()) ? new Button("Re-Open Discussion") : new Button("Mark Resolved");
255
256 //Bind all relevant actions
257 btnClose.setOnAction(e -> viewStage.close());
258 btnReply.setOnAction(e -> onCreateReply());
259 btnEdit.setOnAction(e -> onEdit2());
260 btnDelete.setOnAction(e -> onDelete2());
261 btnMarkResolved.setOnAction(e -> onSolve());
262
263
264 Region spacer = new Region(); //New Spacer
265 HBox actions = new HBox(); //New HBox
266
267
268 //This whole block basically determines what buttons are available based on the solved status, and the users permissions over
269 if (!controller.canEdit(activePost, false, true) && !activePost.getSolved()) actions = new HBox(10, btnReply, btnEdit, btnDele
270 else if (!controller.canEdit(activePost, false, true) && activePost.getSolved()) actions = new HBox(10, btnDelete, spacer, btn
271 else if (controller.canEdit(activePost, false, true) && activePost.getSolved()) actions = new HBox(10, btnMarkResolved, btnDel
272 else if (controller.canEdit(activePost, false, true) && !activePost.getSolved()) actions = new HBox(10, btnReply, btnMarkResol
273
274 //Set Padding things
275 HBox.setHgrow(spacer, Priority.ALWAYS);
276 actions.setPadding(new Insets(10));
277
278 //New Label for a title for the replies section
279 Label replies_Label = new Label("Replies");
280 replies_Label.setFont(Font.font("Arial", 15));
281

```

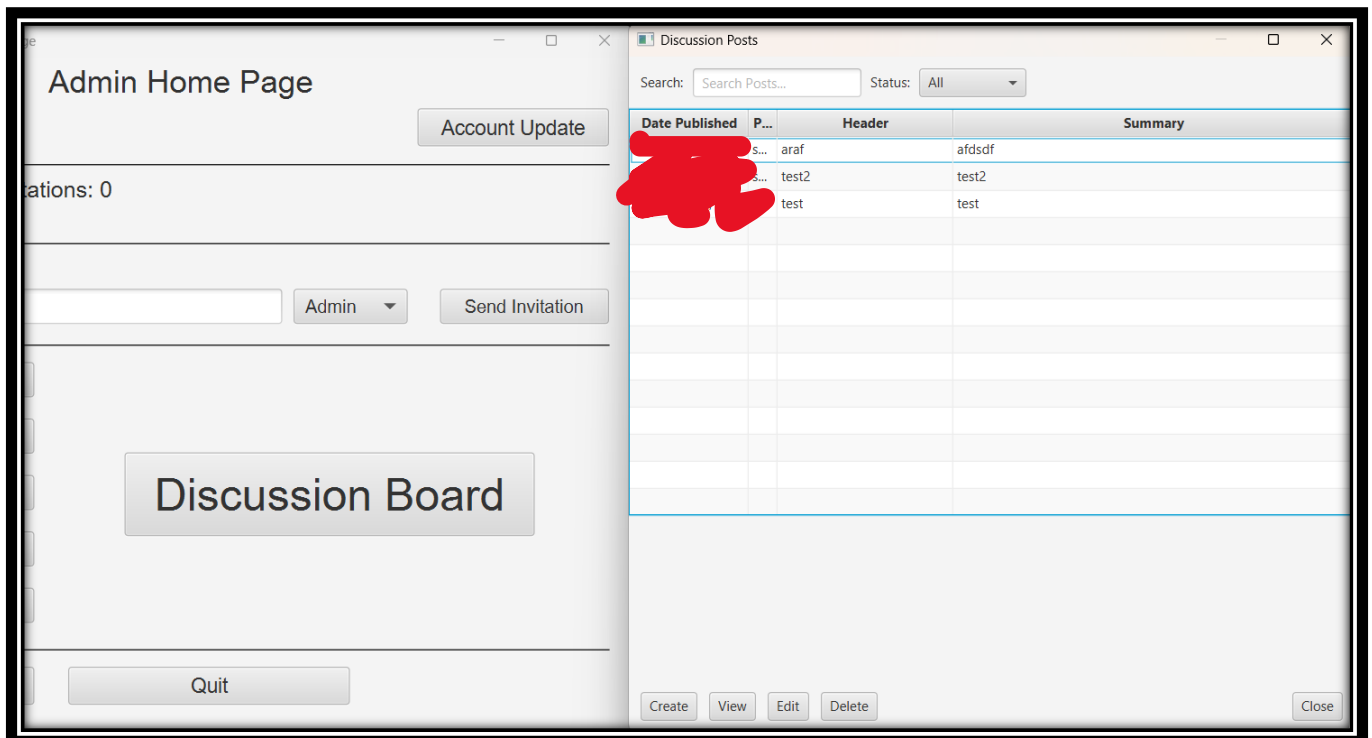
showPost(...) action-region setup: shows reply-window button wiring and the state-based action combinations used when a post is solved or editable.

```

209
210 //This sets it so that the reply content will wrap so that it can be read better
211 //It creates a new type of table cell to be used for the content
212 colBody.setCellFactory(tc -> new TableCell<Reply, String>() {
213
214     //New Text
215     private final javafx.scene.text.Text text = new javafx.scene.text.Text();
216     {
217         text.wrappingWidthProperty().bind(colBody.widthProperty().subtract(16)); //Turn on Text Wrapping
218         setGraphic(text); //Set the cell graphic to text
219         setContentDisplay(ContentDisplay.GRAPHIC_ONLY);
220         setPrefHeight(Region.USE_COMPUTED_SIZE); //Compute the size required for the reply
221     }
222
223     //When needed cell is updated
224     @Override
225     protected void updateItem(String item, boolean empty) {
226         super.updateItem(item, empty); //Normal Cell Function
227
228         //If the item is empty or the string is empty set text to null
229         if (empty || item == null) {
230             text.setText(null);
231             setGraphic(null);
232         }
233         //Else update the graphic normally
234         else {
235             text.setText(item);
236             setGraphic(text);
237         }
238     }
239 });
240

```

Custom TableCell for wrapped reply text: documents the non-obvious UI code that keeps long replies readable in the replies table.



Custom changes for the discussion board – new (DISCUSSION BOARD) button on home page, alongside the discussion board GUI, with all post buttons/interactions.